

PRESENTATION PROBLEMS

Instructor: Orhan Özgüner

© Özgüner @CWRU, 2025

Presentation 1

General Mimetics Corporation uses calibrated Ultralock™ Security Fasteners in the casings of all of its products. To open a product for servicing, a Qualified Technician must first set the General Mimetics Model 301 Duralok™ Security Fastener Adjustment Tool to the Product Code of the fastener used. As this is necessarily a time-consuming and exact process, technicians are encouraged to purchase multiple tools (at the technician's own expense); and keep them set to the most commonly occurring Product Numbers for routine use, changing the setting only when necessary.

Assume that you are able to afford k tools, and keep track of them in an array of codes T where $T[i] = x$ means “Tool number i is set to product code x ”. The day begins with all of the tools set to *NULL*. Each day, you will also be given a work schedule W , listing the product codes of n machines which must be fixed that day (in order). You can tell just by looking if any of the tools is currently set to Code i - we will say that the function `match( $i$ )` returns the index of such a tool (if it exists) or -1 (if it doesn't) in *constant time*.

Devise an efficient algorithm that returns *the minimum number of code changes necessary* to get through the workday. You do not need to return the actual sequence of resets.

Example

Input: $k = 3$, $W = \langle a, b, c, d, b, a, e, d, c, b, e \rangle$

Output: Total Reconfigurations = 6

Explanation:

- Begin by setting the three unset tools to a, b , and c (3 resets).
- Set tool from c to d between jobs 3 and 4.
- Set tool from a to e between jobs 6 and 7.
- Set tool from d to c between jobs 8 and 9.

Presentation 1 Solution

Greedy Choice:

If no tool matches the current machine code W_i , first see if you can set a tool that is currently set to a code you will never need to use again (a code of **NULL**, for instance, will never be used). If there are no such tools, change the tool with the code that you will need to use **furthest in the future**.

Proof of Greedy Choice Property

The optimality of resetting a code that will never be used again, is obvious. So, we will focus on the part of the greedy choice where we replace the code that will be used farthest in the future.

For this proof, we will define a function $\text{future}(a_i, j)$, which takes in a code letter and an index somewhere in W . future returns the index of the *next* occurrence of code a_i after index j , i.e.

$$\text{future}(a_i, j) = \min(k \mid W[k] = a_i \text{ and } k > j)$$

At some index, i , into the work order, let the set of all the codes for which we have matching tools be $A = \{a_1, \dots, a_k\}$. Since we are considering the case where every code occurs somewhere in the future, we cannot have any **NULL** values in A (because **NULL** never occurs). Let a_l be the element in A that is used furthest in the future, i.e.

$$l = \arg \max_{1 \leq j \leq k} (\text{future}(a_j, i))$$

From index i onward, let S be a solution, the list of all tool codes that will be changed, in order, until W is completed. So, $s = \text{length}(S)$ is the minimum number of changes necessary to complete the work order $W[i..n]$. If $S[1] = a_l$, then S is a greedy solution, and we are done. Otherwise, let $S[1] = a_j$ be the code overwritten when working on job i . Suppose the next occurrences of a_j and a_l in the work order are at index t_1 and t_2 , respectively. Based on how we define a_l , it must be true that $t_1 \leq t_2$. So, if we were to form a greedy solution $S' = [a_l, S[2], \dots, S[s]]$, then the number of resets, $s' = \text{length}(S')$, would be $\leq s$ because any future resets due to s_l not being available, occur at the same time or after time t_1 . Therefore, the greedy choice would also produce an optimal solution.

Pseudocode

This algorithm maintains two arrays of size k : one (T) for the codes of all of the current tools, and one (F) for the nearest future occurrence of the code of each tool (a code that will not be used again, has a value of ∞). It proceeds from the start of the workday to the end, checking to see if we have a tool available to match each machine $W[i]$ using the **match** function. If we do not, it performs a replacement. This involves iterating through F to find any ∞ terms, or the highest index of a future occurrence if there are no ∞ terms. The chosen tool $T[j]$ is then reset to $W[i]$. The algorithm then iterates through the rest of W to see in what index (if any) $W[i]$ appears next, and sets $F[j]$ to the corresponding value. Finally, it increments the number of swaps made.

Algorithm 1 Right Tool For The Job

```
1: procedure CALCMAXSWAPS( $W, n, k$ )
2:    $T \leftarrow$  array of length  $k$  filled with NULL
3:    $F \leftarrow$  array of length  $k$  filled with  $\infty$ 
4:    $s \leftarrow 0$ 
5:
6:   for  $i \leftarrow 1$  to  $n$  do
7:     if  $\text{match}(W[i]) = -1$  then
8:        $s \leftarrow s + 1$ 
9:        $\text{toolToReplace} \leftarrow \text{argmax}(F)$ 
10:       $T[\text{toolToReplace}] \leftarrow W[i]$ 
11:      for  $j \leftarrow i$  to  $n$  do
12:        if  $W[j] = W[i]$  then
13:           $F[\text{toolToReplace}] \leftarrow j$ 
14:        end if
15:      end for
16:    end if
17:  end for
18:  return  $s$ 
19: end procedure
```

Runtime Analysis

The two reference arrays T and F are both of size k , so the space complexity of the algorithm is

$$\mathbf{O}(k)$$

The outer for-loop (Line 6) always makes n iterations. Within it, in the worst-case scenario where we perform a replacement every iteration (possible if we work on an entirely new machine every single time), the argmax function on Line 9 can potentially run through every element in F , taking k steps. Then, the inner for-loop (Line 11) can potentially run all the way to the end of W , taking $n - i$ steps each time. So, the total time complexity is

$$\mathbf{O}(n \times k + (n + (n - 1) + (n - 2) + (n - 3) + \dots + 3 + 2 + 1)) = \mathbf{O}(nk + n^2)$$

Presentation 2

You have been tasked with coordinating the sale of payload space aboard the new fleet of nuclear-propelled Orion spacecraft. Companies are offering money to have their experiment payloads launched into the outer solar system; there are many more bids coming in than any one launch can accommodate, so you need to pick the most valuable ones.

The payload bay is a long, thin tube split into numbered modules. Module 1 is located at the nose of the spacecraft, and Module h is located right next to the engines. All of the payloads fit into exactly one module. All of the payloads are also sensitive to heat, vibration, and radiation in varying degrees- since all of these conditions are created by the Orion spacecraft's engines, the payloads have a tolerance expressed as "must be at least *this* many meters away from the engines" or, more succinctly, "must be assigned a module number no greater than t_i ". Module 1 is far enough away from the engines that any single payload could safely be placed there, but the whole bay is not large enough for *every* payload to fit- you will eventually get into a space so close to the engines, that no module can be safely placed there.

More specifically, the n bids are organized in two arrays V and T , where v_i is the offered value of Bid i and t_i is the minimum distance from the engines it can be placed. You want to create a schedule of payloads from Module 1 (at the front) back to the engines, in order to to maximize the total value. This should be in the form of an array S of n non-negative integers, where $S[i]$ represents the position of Payload i . Payloads not being launched can be assigned a value like ∞ , 0, $\emptyset$, or **NULL**

Develop an efficient algorithm to solve this problem.

Example:

- **Input:**

- Number of bids: $n = 4$.
- Bid values: $V = [20, 30, 15, 10]$ million dollars, representing the value v_i for each bid.
- Safety tolerances: $T = [3, 2, 4, 3]$, representing the maximum tolerable module number t_i for each bid.

- **Output:** The optimal order of assigning payloads is $S = [2, 1, 3, \emptyset]$, i.e.:

- First place Payload 2 in slot $1 \leq 2$.
- Then place Payload 1 in slot $2 \leq 3$.
- Then place Payload 3 in slot $3 \leq 4$.
- Payload 4 would have to go in slot $4 > 3$ or higher, violating its safety tolerance, so there is no point including it.

This maximizes the total value of payloads we can fit at 65 million dollars (the value of all the payloads other than Payload Number 4).

Presentation 2 Solution

Greedy Choice

Let i be the most valuable bid, i.e., the bid with the largest v_i . There is at least one optimal solution in which i is given position j , where j is the largest unoccupied module number less than or equal to t_i (i.e. as close to the engines as Payload i can be safely put).

Proof of Greedy Choice Property

Proof. Suppose S , generated by algorithm A , is a module assignment that differs from our greedy assignment S' , generated by algorithm A' . Assume that S and S' first differ at two payloads x and y , where without loss of generality, we will assume that A' examined and placed x , before it placed y . Let v_x and v_y be the values of payloads x and y , respectively.

This implies $v_x > v_y$ due to A' looking at the most valuable remaining bid first. Since $S'[y]$ was empty when y was inserted despite x having been inserted first, this also indicates that $t_x < S'[y]$. Otherwise, x would have been inserted into the then-open space $S'[y]$.

- If it was the case that both x and y were *not* flown in S' , then swapping them causes no change in the total profit: $S[x] = S'[y] = \emptyset$, and $S[y] = S'[x] = \emptyset$. So, S and S' have the same profit.
- If it was the case that both x and y were flown in S' , and both x and y are flown in S , then there is again no change in what bids were accepted. So, S and S' have the same profit.
- If x was flown in S' and was not flown in S , then one of two things can happen.
 - If y was either flown, or not flown, in both S' and S , then we lose the money we made from flying x and don't replace it with anything. S is less profitable than S' .
 - If y was not flown in S but is in S' , then we still lose x . We replace the payload worth v_x , with one worth v_y . $v_y < v_x$, so we have lost money by doing this, and S is again less profitable than S' .
- If try to consider what would happen if x was not flown in S' and x was flown in S , we have to remember that the two variables have switched places: $S[x] = S'[y]$. However, we also know that $t_x < S'[y]$ - so, if we put Payload x in that module, it would be inoperable. No solution S that puts a payload x in a position $S[x] > t_x$ is allowed.

Thus, we have shown that for any non-greedy solution S , there exists a greedy solution S' with an equal or greater profit- the greedy solution is therefore *optimal*. $\square$

Pseudocode

The algorithm that follows from the greedy choice property can be implemented in various ways. We here show an implementation that is efficient and also easy to analyze, but many other solutions are also possible.

The algorithm keeps an array M representing the modules, initially with all empty slots. At each iteration, it considers the next most valuable bid and tries to find an empty slot for it in M before its safety limit (by iterating from the right to left in M , starting from the safety limit). If such a position is found, the algorithm inserts the payload for that slot. Otherwise, the payload is discarded. After that, the algorithm reformats the array from a which-payload-in-which-module format, to a which-module-for-which-payload format S , by assigning $S[M[i]] = i$ in a for-loop.

Algorithm 2 Determine module launch assignments

```

1: procedure ASSIGNMODULES( $V, T$ )
2:    $M \leftarrow$  array of  $\emptyset$  the same length as  $V$ 
3:   Sort  $V$ , and sort  $T$  according to  $V$ .
4:    $n \leftarrow \text{length}(V)$ 
5:
6:   for  $i \leftarrow 1$  to  $n$  do
7:     for  $p \leftarrow T[i]$  down to 1 do
8:       if  $M[p] = \emptyset$  then ▷ If module  $p$  is empty
9:          $M[p] \leftarrow i$ 
10:        break
11:      end if
12:    end for
13:  end for
14:
15:   $S \leftarrow$  array of  $\emptyset$  the same length as  $V$ 
16:   $profit \leftarrow 0$ 
17:  for  $i \leftarrow 1$  to  $n$  do
18:    if  $M[i] \neq \emptyset$  then
19:       $S[M[i]] \leftarrow i$ 
20:       $profit \leftarrow profit + V[i]$ 
21:    end if
22:  end for
23:  return  $S, profit$ 
24: end procedure

```

Runtime Analysis:

This problem involves sorting an array of size n , which can be executed in $\mathbf{O}(n \log n)$ time. It then performs a for-loop that iterates over all the n elements in V , and in each iteration searches over potentially the whole of the n -element-large array M . The final for-loop iterates over the whole of the n -element-large array M , once. The time complexity is thus $\mathbf{O}(n \log n) + n \times n + n = \mathbf{O}(n^2)$.

The program creates one additional array of size n , so it has a space complexity of $\mathbf{O}(n)$.

Presentation 3

You are performing a chemical analysis of an unknown mixture, using a centrifuge to separate its components by their different densities. You have a reference database of the exact weights of all the constituent substances, and you want to match the reference entries to the samples you've centrifuged. Unfortunately, your equipment is not as expensive as what the authors of the database used, so your density measurements come with an observable error range. The error range for each sample i is expressed as a center value d_i , and an error bar value ϵ_i , where the *true* density of the sample could be anywhere between $d_i - \epsilon_i$, and $d_i + \epsilon_i$.

More specifically, the database is given as an array of densities $X[1..n]$, where x_i is the reference density of Substance i . The samples you've weighed have their density measurements given in arrays $D[1..n]$ and $E[1..n]$, where the actual density of Sample j is somewhere between $d_j - \epsilon_j$, and $d_j + \epsilon_j$. You want to match each sample to a database entry based on its density, returning an array M where $M[a] = b$ means "Sample a is Substance b ." If there is more than one possible assignment that fits the evidence, you can return any of them. The same substance will not appear in more than one sample. However, the sample might contain substances that are not in the database, and don't fit in any of the categories. In this case, you should return **NO**.

Devise an efficient algorithm to either identify all the samples, or confirm that no identification is possible. Observe that this is a decision problem, not an optimization problem.

Example:

Input:

- Number of samples and substances: $n = 5$.
- Reference density database: $X = [0.8, 1.5, 2.2, 2.7, 3.4]$.
- Centers of measured sample densities: $D = [2.0, 1.2, 3.3, 2.3, 1.6]$.
- Sample density error factors: $E = [0.4, 0.5, 0.2, 0.9, 0.1]$.

Output:

- A matching exists with $M = [2, 5, 1, 4, 3]$, since:
 - $x_1 = 0.8$ matches with $d_2 = 1.2$ (as 0.8 is within $[0.7, 1.7]$)
 - $x_2 = 1.5$ matches with $d_5 = 1.6$ (as 1.5 is within $[1.5, 1.7]$)
 - $x_3 = 2.2$ matches with $d_1 = 2.0$ (as 2.2 is within $[1.6, 2.4]$)
 - $x_4 = 2.7$ matches with $d_4 = 2.3$ (as 2.7 is within $[1.4, 3.2]$)
 - $x_5 = 3.4$ matches with $d_3 = 3.3$ (as 3.4 is within $[3.1, 3.5]$)
- **However**, if we had $n = 4$, $X = [0.8, 1.5, 2.2, 2.7]$, $D = [2.0, 1.2, 3.3, 2.3]$, and $E = [0.4, 0.5, 0.2, 0.9]$, the answer would be **NO** (no such matching exists) since there is no reference density that is within the window $[3.1, 3.5]$ (but Sample 3 must have a density within this window).

Presentation 3 Solution

Greedy Choice

Let Sample i be the remaining sample with the smallest upper bound for its density range, i.e., $d_i + \epsilon_i$ is the smallest among all the samples. Let Substance x_f be the substance with the lowest density remaining in the database, that could plausibly match Sample i (that is, $d_i - \epsilon_i \leq x_f \leq d_i + \epsilon_i$).

If no such substance exists, return **NO**. Otherwise, match the two ($M[i] = f$).

Proof of Greedy Choice Property

Proof. Assume that there is at least one valid matching for the whole problem. Consider the valid matching S . Assume that x_j is assigned to Sample i in S . We have two cases.

Case 1: $x_j = x_f$. In this case, the valid matching already contains the greedy choice for Sample i , and the proof is done.

Case 2: $x_j > x_f$. In this case, S has to assign x_f to another sample (we will call it Sample k), such that $d_k - \epsilon_k \leq x_f \leq d_k + \epsilon_k$. Now, we will create a different solution S' by swapping these two assignments, i.e., assigning x_f to Sample i and x_j to Sample k .

- We have $d_k - \epsilon_k \leq x_f \leq x_j$ by the definition of x_f , and $x_j \leq d_i + \epsilon_i \leq d_k + \epsilon_k$ by the definition of Sample i , so $x_j \in [d_k - \epsilon_k, d_k + \epsilon_k]$.
- We have $x_f = x_j \in [d_i - \epsilon_i, d_i + \epsilon_i]$ by the definition of x_f .

Thus, S' is also a valid matching for the whole problem. This proves that if there is at least one valid matching, there is also at least one valid matching that assigns x_f to Sample i , complying with the greedy property.

The second part of the claim is straightforward, as Sample i cannot be assigned to any substance in the database, if no substance has a density in its range of possible densities. Therefore, the greedy decision will correctly identify if no valid matching exists. $\square$

Pseudocode

Sort the samples in ascending order of their upper bounds. Sort the database of substances in ascending order of density. This can be any efficient sorting algorithm, like an in-place mergesort. Then, for each sample, find the earliest database entry that fits into the sample's confidence interval. Assign that substance to the sample, and move on to the next sample. If we fail to find a suitable match, we know that no valid matching exists.

Algorithm 3 Sort Chemical Samples with Greedy Approach

```
1: procedure FINDMATCHING( $X, D, E, n$ )
2:   Sort  $D$  and  $E$  by  $d_x + \epsilon_x$  in ascending order.
3:   MERGE-SORT( $X$ ) ▷ Sort database by density
4:
5:    $M \leftarrow$  Empty array of size  $n$ 
6:   for  $i \leftarrow 1$  to  $n$  do
7:     for  $j \leftarrow 1$  to  $n$  do
8:       if  $D[i] - \epsilon[i] \leq x_j \leq D[i] + \epsilon[i]$  then
9:          $M[i] \leftarrow j$ 
10:         $X[i] \leftarrow \infty$  ▷ Prevent  $X[i]$  from being matched twice.
11:        break
12:      end if
13:    end for
14:    if  $M[i]$  is still empty then
15:      return NO
16:    end if
17:    return  $M$ 
18:  end for
19: end procedure
```

Runtime Analysis

The worst-case runtime occurs when there is a correct matching and all samples in D can be assigned. The program may terminate early if it encounters a sample that cannot be assigned. In the worst-case scenario, each $X[j]$ might have to be compared with all possible samples before finding a suitable match at the very end of the un-matched region (marking the matched values in X will cause this region to shrink by 1 each time).

Since there are n samples and n substances in the database, in the worst case, we perform as many checks as there are unmatched x_i s, leading to a quadratic time complexity:

$$T(n) = \sum_{i=1}^n i = \frac{n(n-1)}{2} = \mathbf{O}(n^2)$$